

# School Management & Academic Information Web Portal

## *Proposed System Architecture, Technology Stack, Modules & Team Responsibilities*

Prepared as a development planning document for the five-member project team.

### 1. Project Overview

The proposed system is a centralized web-based school management and academic information portal connecting the Ministry of Education, schools, teachers, students and parents. The system will support school administration, user management, timetables, attendance, academic results, report cards, parent engagement and notifications.

Primary user hierarchy:

- Ministry System Administrator
- School System Administrator
- Teacher
- Student
- Parent

Additional school staff such as school heads, deputy heads, HODs and administrative staff may be represented as secondary school users with appropriate permissions, without making them core roles in the first version.

### 2. Recommended Technology Stack

| Area             | Recommended Technology                     | Purpose                                                       |
|------------------|--------------------------------------------|---------------------------------------------------------------|
| Frontend         | React + JavaScript/TypeScript + HTML + CSS | Interactive, component-based user interface                   |
| UI               | Bootstrap or Tailwind CSS                  | Responsive layouts, forms, dashboards and components          |
| Backend          | Java + Spring Boot                         | Business logic, REST APIs, authentication and system services |
| Security         | Spring Security                            | Authentication, authorization and role-based access control   |
| Persistence      | Spring Data JPA + Hibernate                | Database access and object-relational mapping                 |
| Database         | PostgreSQL                                 | Persistent storage for system data                            |
| Database Tool    | pgAdmin / DBeaver                          | Database administration and SQL work                          |
| Build Tool       | Maven                                      | Java dependency and project management                        |
| IDE              | IntelliJ IDEA                              | Java/Spring development                                       |
| Frontend Tooling | Node.js + npm                              | React development and package management                      |

|                 |                                            |                                                                 |
|-----------------|--------------------------------------------|-----------------------------------------------------------------|
| Version Control | Git + GitHub                               | Team collaboration and source-code history                      |
| API Testing     | Postman                                    | Testing REST endpoints                                          |
| Diagrams        | diagrams.net (Draw.io)                     | ERD, UML, architecture and process diagrams                     |
| Testing         | JUnit + Mockito + Postman                  | Unit and API/integration testing                                |
| Charts          | Chart.js / Recharts                        | Attendance, performance and engagement dashboards               |
| Reports         | OpenPDF or a Spring-compatible PDF library | Generating downloadable report cards                            |
| Notifications   | Email service + SMS gateway/API            | Parent/student notifications and SMS for non-smartphone parents |

### 3. React vs Plain HTML/CSS/JavaScript

React is not a replacement for HTML, CSS and JavaScript. React is a JavaScript library for building interfaces using reusable components. React applications still ultimately render HTML and use CSS for styling.

| Option                  | Advantages                                                                                                                                         | Disadvantages                                                                                 |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| HTML + CSS + JavaScript | Simple setup; easy to understand; fewer dependencies; suitable for small/static sites.                                                             | As the portal grows, repeated UI code and complex state management become harder to maintain. |
| React                   | Reusable components; strong support for dashboards/forms; easier management of complex interactive interfaces; good separation of UI into modules. | Requires Node.js/npm and React-specific concepts; more initial setup and learning.            |

Recommendation: use React for the frontend because this project has multiple role-specific dashboards, forms, tables, attendance workflows, results screens, notifications and reusable UI components. The backend should remain a separate Spring Boot application exposing REST APIs. For a five-person team, React gives a useful structure without requiring the complexity of a full microservices architecture.

### 4. Proposed High-Level Architecture

The recommended architecture is a modular monolithic backend with a separate React frontend. Avoid microservices for the first project version; they add deployment, networking and maintenance complexity that is unnecessary for the academic scope.

- User Browser → React Frontend
- React Frontend → Spring Boot REST API
- Spring Boot → Spring Security for authentication/authorization
- Spring Boot → Service Layer for business rules
- Service Layer → Spring Data JPA/Hibernate
- JPA/Hibernate → PostgreSQL Database
- Spring Boot → Email/SMS services where required

## 5. Core System Roles and Responsibilities

### Ministry System Administrator

- Create, edit, activate and deactivate schools.
- Manage school-level administrators.
- View Ministry-wide statistics and reports.
- Monitor registered schools and high-level system activity.

### School System Administrator

- Manage users belonging to the school.
- Add/manage teachers, students, parents and relevant school staff.
- Manage school classes, subjects, academic years and terms.
- Manage or approve school timetables.
- Verify and publish results.
- Manage/reset user accounts and school-issued credentials.
- Monitor attendance, results and parent engagement.

### Teacher

- View assigned classes and subjects.
- Provide availability/teaching information for timetable planning.
- Create or start lesson attendance sessions.
- Record attendance manually or use a temporary attendance code.
- Enter and submit student marks.
- View relevant student academic and attendance information.

### Student

- Log in using a school-issued account.
- View timetable.
- Participate in attendance using a temporary class code where available.
- View attendance.
- View and download results/report cards where permitted.
- View relevant school notifications.

### Parent

- View linked child/children.
- View attendance, timetable and academic results.
- Download report cards.
- Receive notifications.
- Receive SMS notifications when the parent is registered as not having smartphone access.

## 6. Important Business Rules

- A school must first be created by the Ministry System Administrator before a School System Administrator can manage it.
- A School Administrator can only manage data belonging to their assigned school.

- A user must have an appropriate role before accessing role-restricted functions.
- Teachers can enter and submit marks, but final publication should be controlled by the School Administrator or another authorized school authority.
- Students should be able to view their own academic information only.
- Parents should only see children linked to their account.
- A teacher should only manage attendance and academic information for classes/subjects assigned to them.
- Attendance codes should be temporary rather than permanent to reduce abuse and sharing.
- Teachers should be able to manually mark attendance for students who cannot use the digital attendance method.
- Parent engagement tracking should record relevant events such as report publication and report viewing.
- School data must be isolated so that one school cannot access another school's student, teacher or parent data.

## 7. Main System Modules

- Authentication and Authorization
- Ministry School Management
- School Management
- User and Account Management
- Student Management
- Teacher Management
- Parent Management
- Subject and Class Management
- Timetable Management
- Attendance Management
- Grade/Mark Management
- Results and Report Card Management
- Parent Engagement Monitoring
- Notifications
- SMS Integration
- Reporting and Analytics
- Audit and Security Logging

## 8. Attendance Workflow

1. Teacher selects an assigned class and starts a lesson/attendance session.
2. The system records the class, subject, teacher and time.
3. The teacher can manually mark students present/absent.
4. Alternatively, the system can generate a temporary attendance code for the lesson.
5. Students enter the code during the allowed time window.
6. The system records the attendance event with student, class, date and time.
7. Students without suitable devices can be marked manually by the teacher.

## 9. Results Workflow

8. Teacher enters student marks.
9. Teacher reviews and submits marks.
10. Authorized school staff verify the submitted results.
11. School System Administrator publishes the approved results.
12. Students can view their results.
13. Parents can view and download the linked child's report.
14. The system records publication and relevant parent viewing activity.

## 10. Parent Engagement and SMS

The system can record whether a parent has accessed a published report. This can be used by authorized school staff to identify parents who may need follow-up. The system should avoid exposing unnecessary parent activity information to users who do not need it.

During account creation, the School Administrator can record whether the parent has smartphone/internet access. For parents registered as SMS-only, the system can trigger an SMS notification when important information such as results is published. The first development version should use a mock SMS service or log SMS requests before integrating a paid/real SMS gateway.

## 11. Proposed Database

PostgreSQL is a strong choice for this project and can be used instead of MySQL. It is a mature relational database and works very well with Spring Boot, JPA/Hibernate and structured school data.

Initial database entities/tables may include:

- users
- roles
- user\_roles
- schools
- school\_admins
- teachers
- students
- parents
- parent\_students
- school\_staff
- subjects
- classes
- class\_students
- teacher\_subjects
- teacher\_classes
- academic\_years
- terms
- timetables
- attendance\_sessions

- attendance\_records
- grades
- results
- report\_cards
- notifications
- sms\_logs
- parent\_activity
- audit\_logs

The final schema should be designed from the requirements and ERD rather than creating all tables at once. The team should agree on primary keys, foreign keys, cardinalities, constraints and indexes before implementation.

## 12. Development Tools Everyone Should Install

- JDK 21
- IntelliJ IDEA
- Maven
- Git
- GitHub account
- Node.js (LTS) and npm
- PostgreSQL
- pgAdmin 4 or DBeaver
- Postman
- diagrams.net / Draw.io
- A modern browser such as Chrome, Edge or Firefox

## 13. Spring Boot Dependencies

- Spring Web
- Spring Security
- Spring Data JPA
- Validation
- PostgreSQL Driver
- Thymeleaf is NOT required if React is chosen as the frontend.
- Spring Boot DevTools (development convenience)

If the team chooses React, the frontend should be a separate project. The backend should expose REST endpoints and should not depend on server-rendered Thymeleaf pages.

## 14. React Frontend Structure

- src/components – reusable UI components
- src/pages – role-specific pages/screens
- src/layouts – dashboard and navigation layouts
- src/services – API calls

- src/hooks – reusable React logic
- src/context or state – authentication/user state where needed
- src/utils – helper functions
- src/assets – images and other static assets

Suggested role-based page groups:

- Ministry Dashboard
- School Administration Dashboard
- Teacher Dashboard
- Student Dashboard
- Parent Dashboard

## 15. Spring Boot Backend Structure

- controller – REST endpoints
- service – business logic
- repository – database access
- entity – JPA entities
- dto – request/response objects
- security – authentication and authorization
- exception – error handling
- config – application configuration
- util – shared utilities

## 16. Suggested Five-Person Team Division

### Member 1 – Frontend/UI Lead

React components, dashboards, forms, tables, responsive UI, navigation and frontend integration.

### Member 2 – Backend & Security Lead

Spring Boot foundation, REST APIs, authentication, Spring Security, roles, permissions and account management.

### Member 3 – Academic/Attendance Lead

Classes, subjects, teachers, timetable workflows and attendance.

### Member 4 – Database & Results Lead

PostgreSQL schema, ERD, JPA entities, grades, results, report cards and database quality.

### Member 5 – Parent/Integration/QA Lead

Parent portal, parent engagement, notifications/SMS integration, testing, integration and GitHub coordination.

The role division should not create isolated developers. Everyone should understand the full request flow from React → REST Controller → Service → Repository/JPA → PostgreSQL and should review each other's code.

## 17. GitHub Collaboration Strategy

- Create one shared GitHub repository.
- Protect the main branch.
- Use a development branch or pull-request workflow.
- Create feature branches for individual modules.
- Use descriptive commit messages.
- Open pull requests before merging significant changes.
- Review each other's code.
- Keep environment secrets such as database passwords and API keys out of GitHub.
- Maintain a README containing setup instructions.

## 18. Recommended Development Order

15. Finalize requirements, user roles and permissions.
16. Create use cases and define the MVP.
17. Design the ERD and database relationships.
18. Design the high-level system architecture.
19. Create the GitHub repository and project conventions.
20. Create the Spring Boot backend and PostgreSQL connection.
21. Create the React frontend and establish API communication.
22. Implement authentication and role-based access.
23. Implement Ministry and School management.
24. Implement teacher, student and parent management.
25. Implement classes, subjects and timetables.
26. Implement attendance.
27. Implement grades and result approval/publication.
28. Implement parent portal and report downloads.
29. Implement parent engagement tracking and notifications.
30. Add SMS integration or a mock SMS service.
31. Perform unit, API, integration, security and usability testing.
32. Prepare deployment, documentation, diagrams and final demonstration.

## 19. Minimum Viable Product (MVP)

To control project scope, the team should first complete the following minimum system:

- Secure login and role-based dashboards.
- Ministry Administrator creates and manages schools.
- School Administrator manages teachers, students and parents.
- Teacher manages assigned classes and records attendance.
- Teacher enters and submits grades.
- School Administrator approves/publishes results.
- Students view timetable, attendance and results.
- Parents view linked children's timetable, attendance and results.
- Parents can download a report card.
- Basic parent report-view tracking.



SMS, advanced analytics, sophisticated timetable optimization and other integrations should be treated as secondary features unless the academic requirements explicitly make them mandatory.

## 20. Immediate Team Action Plan

- Agree on the final project title and problem statement.
- Confirm the five user roles and permissions.
- Turn each role into a list of functional requirements.
- Agree on the MVP and clearly define what is out of scope.
- Choose React + Spring Boot + PostgreSQL as the working stack.
- Install the standard development tools on all five computers.
- Create the GitHub repository.
- Create the first ERD draft.
- Create the use case diagram.
- Use the agreed requirements as the basis for the 28 August submission.

## 21. Final Recommendation

Recommended stack: React for the frontend, Java/Spring Boot for the backend, PostgreSQL for the database, Git/GitHub for collaboration, IntelliJ IDEA for Java development, Node.js/npm for React, Postman for API testing, and Draw.io for system diagrams. This gives the team a modern, industry-relevant architecture while remaining manageable for a five-person academic project.

The key architectural principle is separation of concerns: React handles the user interface; Spring Boot handles business rules and security; PostgreSQL stores persistent data; external services handle SMS/email; and GitHub coordinates the team's source code.